基于 FPGA 的多模式信号发生器

摘要

本项目基于 AMD-Xilinx Artix-7 FPGA，打造了一款“底座+多模式”的可扩展信号发生平台。系统以 32 位 DDS（直接数字频率合成）内核为基础，支持正弦、方波、三角、锯齿四款基础波形，深度融合了 LFSR 伪随机跳频、周期扫频、AM/FM 模拟调制以及 CPFSK 数字调制等多种通信模式。项目配套开发了基于 Python 的可视化上位机，不仅支持波形实时回传显示，还创新性地实现了“鼠标手绘任意波形，通过双口 RAM 实时下载至 FPGA 输出”的软硬件协同设计。系统在 100MHz 时钟下时序完美收敛，实测动态功耗仅 0.206W，资源占用极低，是一套兼具通信深度与人机交互的工程实践。

项目背景

1. 选题原因

在高校电子信息、通信工程的教学与基础科研中，信号发生器是不可或缺的核心仪器。然而，传统的商用信号发生器（如台式射频信号源）价格昂贵、体积庞大，难以做到人手一台；市面上常见的廉价 DDS 模块又往往只能输出单一的正弦/方波，且频率调节依赖实体旋钮或死板的按键，缺乏直观的人机交互与任意波形生成（AWG）能力。本项目致力于打破这一痛点，利用 FPGA 强大的并行处理能力，打造一款低成本、高集成、且具备高度可视化交互的“口袋实验室”仪器，让使用者能通过电脑直观地“画”出自己想要的任意波形。

2. 前人经验

目前基于 FPGA 的信号发生器方案主要分为两类：

基础 DDS 方案：多数开源项目和初学者作品仅实现了 32 位相位累加器与基础波形 ROM 查找表。这类方案虽然能输出正弦波，但功能单一，缺乏 AM/FM 模拟调制与 CPFSK 等通信级数字调制，无法满足现代通信实验的需求。

软硬件割裂方案：部分带上位机的方案，通常只是通过串口发送频率参数。当需要输出自定义波形（AWG）时，通常需要事先在 PC 端生成 .mif 或 .coe

文件，重新用 Quartus/Vivado 综合并烧录整个固件。这种“修改波形=重新编译”的流程极其繁琐，完全丧失了对信号生成的实时性与可交互性。

3. 本项目的破局思路

基于上述不足，本项目在传统 DDS 底座上进行了深度重构。一方面，将跳频、扫频、AM/FM、CPFSK 等多种通信模式在底层硬件解耦集成，实现多模式的无缝切换；另一方面，引入双口 RAM（Dual-Port RAM）架构，打通了 Python 上位机与 FPGA 的高速数据通道。使用者只需在 PC 上用鼠标手绘轨迹，数据即可实时下发至 FPGA 硬件内部，彻底告别了“改波形要重编译”的行业痛点，实现真正的软硬协同。

复现说明

1. 打开 Vivado 2018.3，在 Tcl Console 中运行：`source 你的文件路径/build/build_project.tcl`

2. 完成后会进入主页面，请等待 1 分钟后再进行下一步。

3. 点击`Run Synthesis`，可能会出现 error，这时重新点击`Run Synthesis`，直至 error 消失只剩 warnings，完成之后点击`Run Implementation`，等待完成。

4. 等待工程构建完成后，点击 `Generate Bitstream`。

5. 点击`Open Hardware Manager`栏下的`Program Device`,选择开发板,将`.bit` 文件下载至开发板。

6. 将开发板通过 USB 连接至 PC，确认设备管理器中出现串口（如 COM5）。

7. 安装 Python 依赖：`pip install pyserial matplotlib numpy`。

8. 运行 `board/dds_host.py`，选择对应串口，波特率设为 `921600`，点击“打开串口”即可交互。

第一部分 作品概述

1.1 功能与特性

本系统具备高度集成的信号发生能力。基础功能上，支持 0~1000Hz 频率范围的正弦、方波、三角、锯齿波输出；高级通信模式上，内置扫频模式、LFSR 跳

频模式、AM/FM 调制模式以及连续相位的 CPFSK 频移键控模式。交互特性上，支持按键本地控制以及串口上位机远程控制。特色功能为“手绘任意波形发生器（AWG）”，用户在上位机界面用鼠标绘制波形轨迹，系统即可通过 921600 bps 的波特率将数据高速写入 FPGA 的双口 RAM 中，实现“所见即所得”的任意波形输出。

1.2 应用领域

本作品可作为高校电子信息工程、通信工程等专业的低成本“口袋实验室”设备，用于验证数字信号处理、模拟调制、数字调制及通信原理。同时，其极简的硬件架构与高效的双口 RAM 下发机制，可直接复用于工业电机控制波形生成、超声波驱动信号发生以及自动化测试设备（ATE）的激励源模块中。

1.3 主要技术特点

采用模块化、全解耦的 RTL 架构设计；32 位高精度相位累加器确保了极高的频率分辨率；双口 RAM 采用独立时钟的读写分离方案解决了“上位机写数据”与“DDS 读数据”的冲突；自定义轻量级串口协议（帧头+数据+帧尾）保障了通信的稳定可靠。

1.4 主要性能指标

指标类别	指标名称	参数/状态
基础波形	最佳平滑显示范围	0.1~200Hz 曲线最平滑，
	失真临界点	800Hz 开始折线感明显
扫频模式	扫频范围与步进	1Hz~1000Hz，步进 10Hz
跳频模式	随机跳频范围	1Hz~1000Hz
模拟调制	AM/FM 载波与频偏	载波范围：46.5Hz ~1164 Hz
		调制信号：内部低频三角波
数字调制	CPFSK 调制参数	双频点：500Hz (逻辑 0) / 1000 Hz (逻辑 1)；
		传输速率：100 bps

手绘 AWG	任意波形生成	采样点：1024 点(12bit)；全量下发耗时：约 2.0 秒
底层硬件	核心时序与资源	100 MHz 时钟(PLL 倍频)；WNS +0.226 ns；LUT 2186 (11%)；动态功耗 0.206 W

1.5 主要创新点

1.“底座+多模式”高可重构架构：以 32 位 DDS 为底座，模块化解耦并融合了 LFSR 跳频、扫频、AM/FM 模拟调制与 CPFSK 数字调制，实现了单一硬件平台下多种通信模式的无缝切换。

2. 软硬协同的手绘任意波形实时下发：开发 Python 可视化界面捕获鼠标轨迹，FPGA 内部采用双口 RAM（读写分离），解决了串口通信写数据与 DDS 读数据的速度冲突，实现任意波形生成。

3. 低功耗的工程实现：通过位宽截断与硬件 DSP 乘法器推断，系统在 100MHz 时钟下时序收敛（WNS +0.226ns），动态功耗仅 0.206W，LUT 资源占用仅 11%，为扩展留足空间。

1.6 设计流程

1.需求分析与架构设计：确定项目功能，完成上位机与 FPGA 的软硬件功能划分。

2.RTL 代码编写与仿真：使用 Verilog HDL 编写 DDS 核心及各调制模块，之后进行时序仿真与功能验证。

3.综合实现与时序优化：通过 Vivado 进行综合与布局布线，添加时钟约束，优化关键路径，确保 100MHz 时序收敛。

4.上位机软件开发：基于 Python 开发 GUI 界面，实现串口多线程通信与波形动态绘制。

5.板级联调与闭环验证：结合 Vivado ILA 逻辑分析仪与上位机进行物理层抓包，验证系统全链路闭环功能。

第二部分　系统组成及功能说明 2.1 整体介绍

本系统由上位机（Python 软件）与下位机（Artix-7 硬件系统）构成。上位机负责界面渲染、鼠标轨迹转数字序列、串口打包下发及接收波形回传。下位机负责核心信号运算与输出。

各模块之间的关系：用户输入指令经 UART 解析后，改变系统的状态机或写入数据至双口 RAM，DDS 核心根据当前状态产生波形，最后通过数字引脚输出，同时数据打包模块将信号回传至上位机进行波形显示。

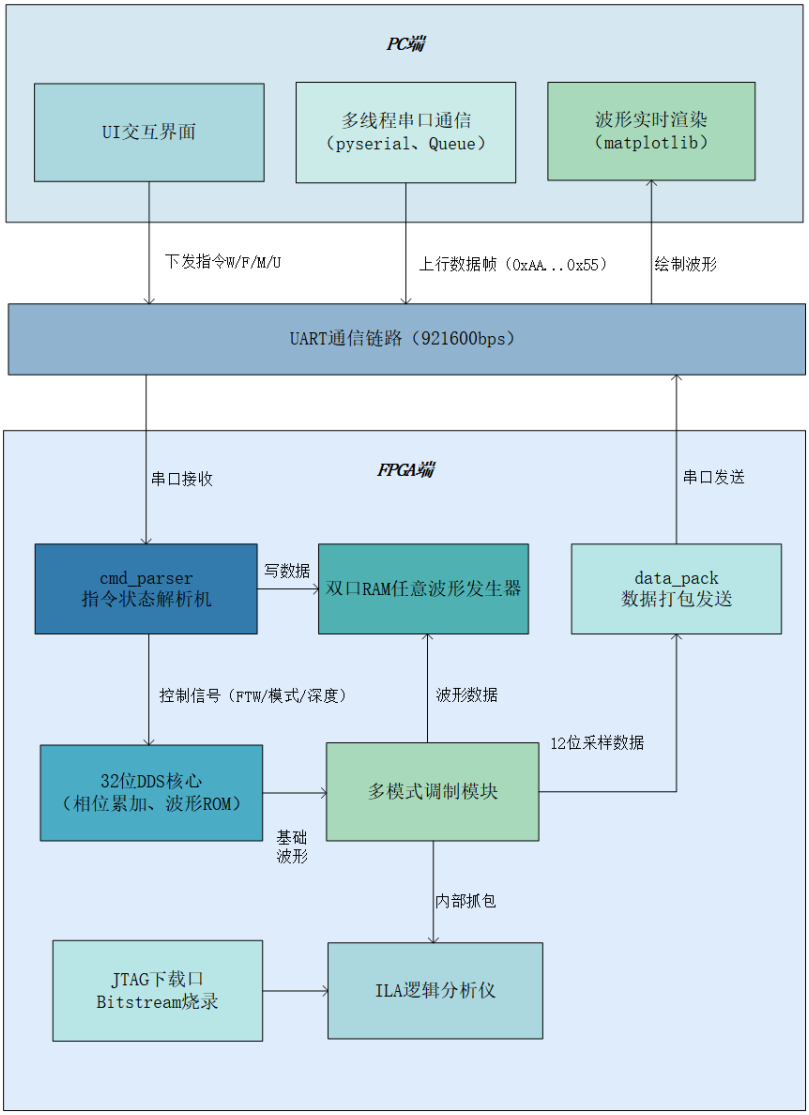


图 2-1　系统整体架构与数据流图

2.2 硬件系统介绍

2.2.1 硬件整体介绍

本系统硬件平台采用正点原子达芬奇开发板，其主控核心为 AMD-Xilinx

Artix-7 系列芯片 XC7A35T-2FGG484。硬件系统主要由以下部分组成：

1. 核心处理单元：XC7A35T 芯片，内含 33,280 个逻辑单元及丰富的 Block RAM 资源，负责所有信号发生算法的硬件逻辑实现。

2. 时钟与复位系统：采用板载 50MHz 有源晶振作为系统外部时钟源，经 FPGA 内部 PLL（锁相环）倍频至 100MHz 作为全局工作时钟；使用板载复位按键提供系统异步复位信号。

3. 人机交互外设：引出了 4 个板载独立按键（KEY1~KEY4）用于本地波形切换与频率步进控制。

4. 下载与调试链路：使用 10-pin 外置 JTAG 下载器连接 PC 与开发板，用于 Bitstream 的烧录以及 Vivado 内部 ILA 逻辑分析仪的在线波形抓取。

5. 串口通信链路：通过板载 USB 转串口芯片，以 921600 bps 波特率与 PC 端上位机建立全双工通信，实现控制指令下发与波形数据回传。

2.2.2 机械设计介绍

无。

2.2.3 电路各模块介绍

1. 电源与时钟模块：开发板通过外部 5V 电源供电；时钟模块利用板载 50MHz 晶振输入至 FPGA 专用时钟引脚，经内部 PLL 模块产生 100MHz 全局时钟。

2.物理通信接口（UART）：通过引脚约束（XDC），将 FPGA 的 uart_rx 与 uart_txd 绑定至板载 USB 转串口芯片的对应引脚，实现指令与数据交互。

3.下载与调试接口（JTAG）：利用 10-pin JTAG 接口连接外部下载器，引出 TCK、TMS、TDI、TDO 等引脚，用于 Bitstream 烧录及 ILA 抓包。

4.核心逻辑电路（RTL 级设计）：系统的“电路”主要体现在 FPGA 内部的硬件网表。核心由 32 位 DDS 相位累加器电路、双口 RAM 存储阵列电路、UART 串行收发状态机电路以及各调制乘法器（DSP 资源）电路协同构成。

```
create_clock -period 20.000 -name clk_50M [get_ports clk_50M]

# 时钟引脚
set_property -dict {PACKAGE_PIN R4 IOSTANDARD LVCMOS15} [get_ports clk_50M]
set_property CLOCK_DEDICATED_ROUTE FALSE [get_nets clk_50M]
# 复位引脚
set_property -dict {PACKAGE_PIN U7 IOSTANDARD LVCMOS15} [get_ports rst_n]

# UART 接收引脚
set_property -dict {PACKAGE_PIN E14 IOSTANDARD LVCMOS33} [get_ports uart_rx]
set_false_path -from [get_ports uart_rx]

# 按键引脚
set_property -dict {PACKAGE_PIN T4 IOSTANDARD LVCMOS15} [get_ports {key[0]}]
set_property -dict {PACKAGE_PIN T3 IOSTANDARD LVCMOS15} [get_ports {key[1]}]
set_property -dict {PACKAGE_PIN R6 IOSTANDARD LVCMOS15} [get_ports {key[2]}]
set_property -dict {PACKAGE_PIN T6 IOSTANDARD LVCMOS15} [get_ports {key[3]}]

# uart_txd
set_property -dict {PACKAGE_PIN D17 IOSTANDARD LVCMOS33} [get_ports uart_txd]
```

图 2-2 系统物理引脚约束配置

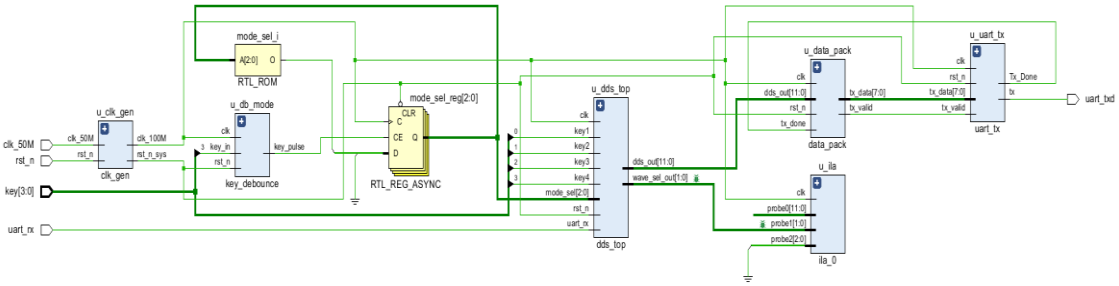


图 2-3 系统 RTL 顶层逻辑网表

2.3 软件系统介绍

2.3.1 软件整体介绍

PC 端上位机采用 Python 3.12.7 开发，基于 tkinter 构建图形用户界面，结合 pyserial 实现高速串口通信，并利用 matplotlib 完成示波器级别的波形动态绘制。软件采用多线程架构，将 UI 渲染与底层串口数据接收解耦，确保在高频数据回传时界面不卡顿。软件主要由串口配置、模式切换、波形实时显示、鼠标手绘输入以及日志记录五个模块构成。

2.3.2 软件各模块介绍

1. 通信与多线程管理模块：

底层基于 pyserial，支持端口自动扫描与 921600 波特率连接，我们采用守护线程结合队列进行数据接收。线程中实现了一套状态机，严格校验 0xAA 帧

头、0x55 帧尾，提取 12 位采样数据（高 4 位+低 8 位），避免数据粘包导致解析崩溃。

2. 基础控制模块：

支持一键下发波形切换（W0~W3）、频率设置（自动计算 32 位 FTW 并下发 F 指令）、扫频与调制参数配置等。

3. 手绘任意波形（AWG）模块：

用户在 Canvas 画布上按下鼠标拖动时，程序实时捕获 X/Y 坐标，将其归一化映射为 0~4095 的 12 位有效数据（1024 个点）。

点击“上传到 FPGA”后，软件自动先发送 M5 进入 AWG 模式，然后以 2ms 的间隔逐条下发 U{addr},{data} 指令，写入 FPGA 的双口 RAM 中。

4. 波形实时回显模块：

接收来自硬件 data_pack 模块上传的数据帧。为了兼顾流畅度，软件对刷新频率进行了节流处理，将 0~4095 的数字量转为 -1~1 的浮点数，在深色示波器风格的画布上实时绘制曲线。

5. 日志与状态记忆模块：

带有时间戳的收发日志记录，支持导出为本地 txt 文件；关闭软件时，自动通过 JSON 格式记忆波特率、上次连接的端口和频率。

第三部分　完成情况及性能参数

3.1　整体介绍

本系统最终以正点原子达芬奇开发板为核心物理载体，搭配 PC 端上位机形成完整的交互系统。硬件实物连接包含：10-pin JTAG 下载线（用于程序烧录与调试）、USB 串口通信线（用于上位机数据交互）。

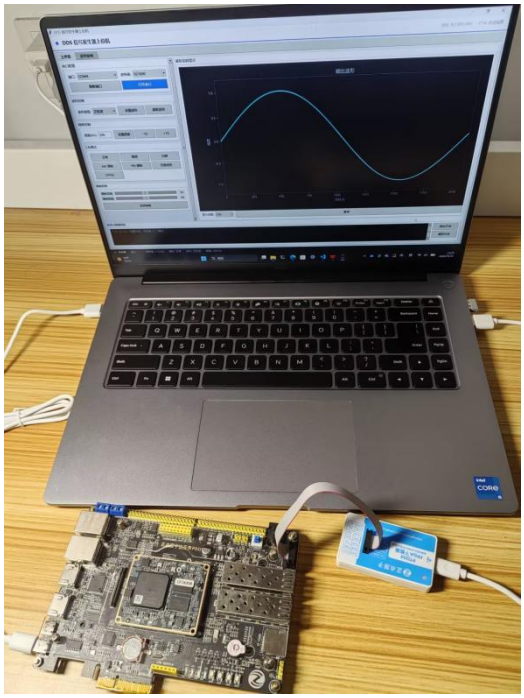


图 3-1 系统实物正面

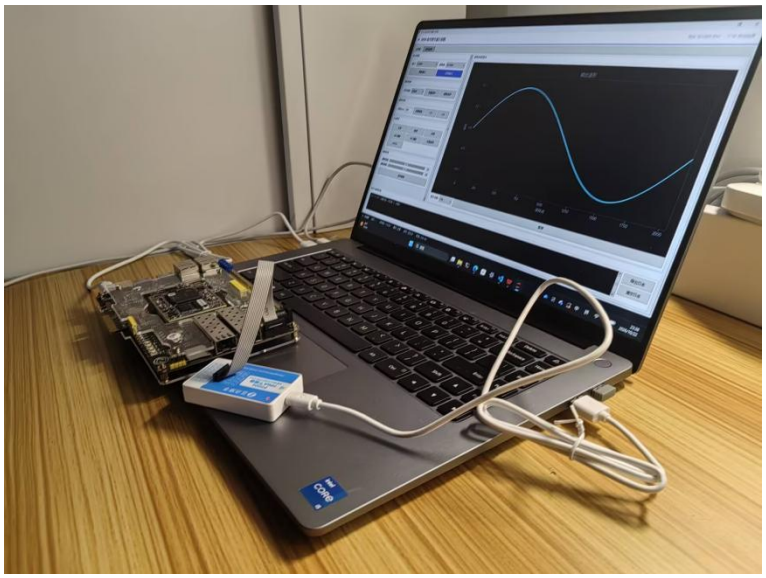


图 3-2 系统实物斜向 45°

3.2 工程成果

3.2.1 机械成果

本作品属于纯 FPGA 逻辑设计与信号处理验证平台，未涉及外部机械结构设计，故无机械成果。

3.2.2 电路成果

本作品基于成熟开发板，未进行额外底层 PCB 制板。系统的电路工程成果

已体现在第二部分的 RTL 顶层逻辑网表及引脚约束中。

3.2.3 软件成果

本项目的软件系统（PC 端界面与 FPGA 内部固件）均已开发完成并成功联调，核心运行界面展示如下：

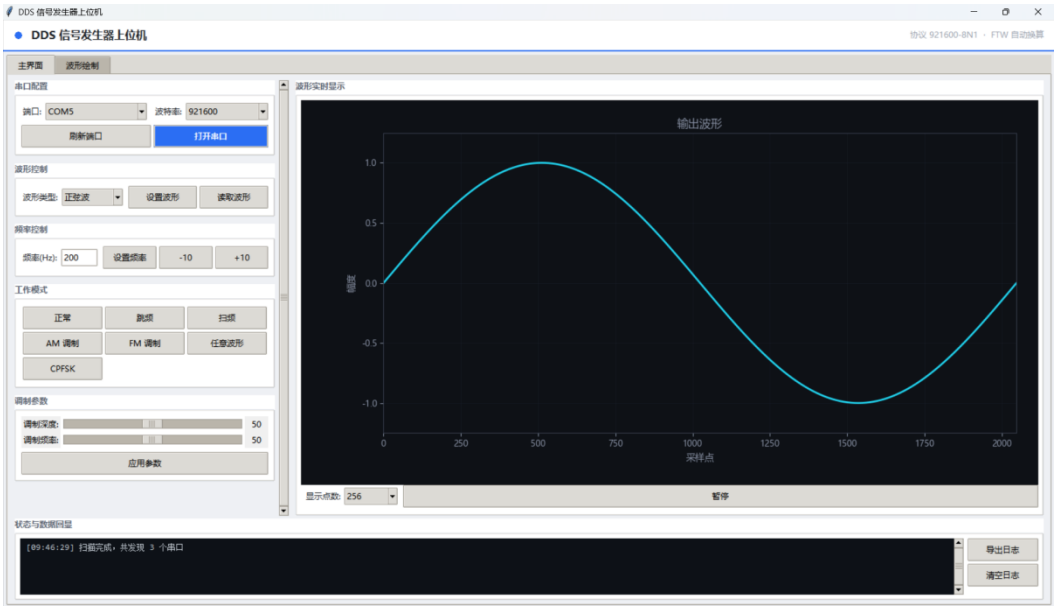


图 3-3 上位机界面

3.3 特性成果

系统实测性能与功能均已达标，核心量化指标如下：

1. 时序与功耗表现：100MHz 时钟完美收敛，WNS 为 +0.226ns，动态功耗仅 0.206W。

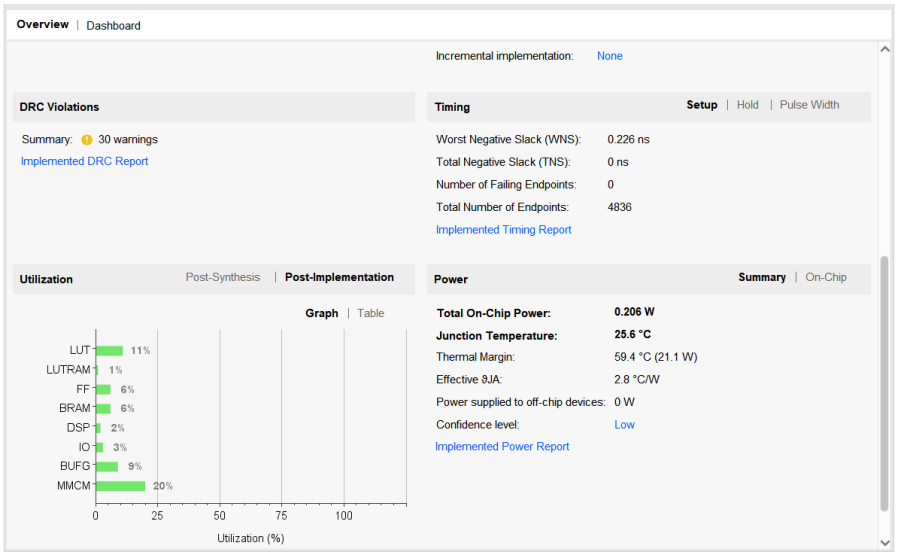


图 3-4 时序与功耗

2. 系统物理抓包验证：通过 ILA 逻辑分析仪抓取内部总线，验证多模式波形实时输出正常。

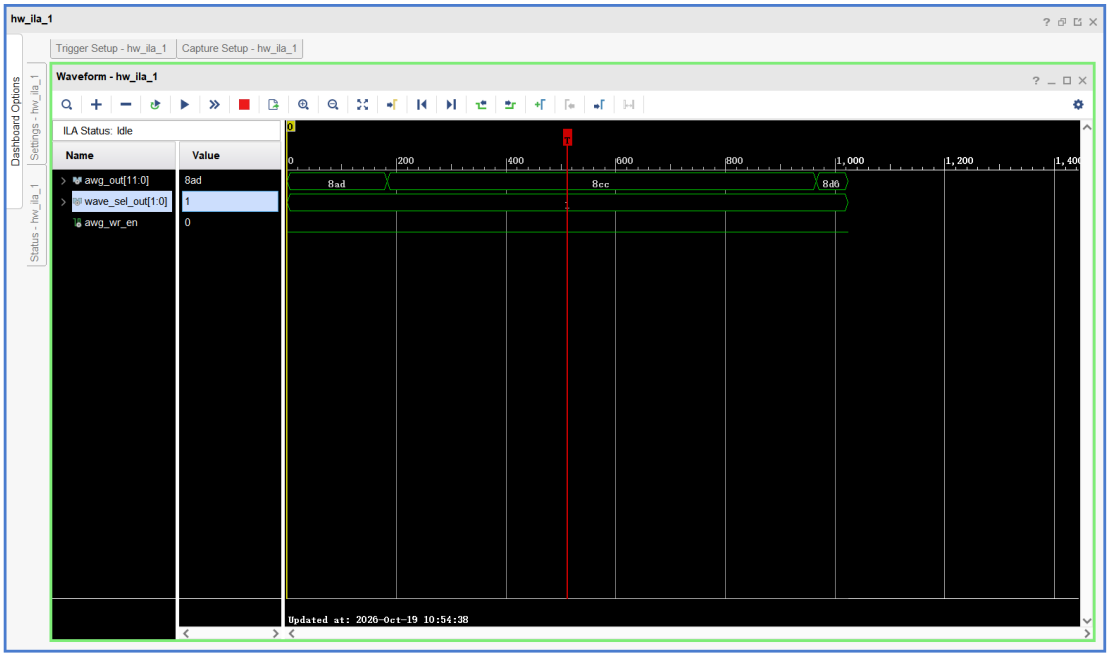


图 3-5 ILA 验证

第四部分 总结 可扩展之处

未来本平台可在三个维度继续演进：一是硬件链路升级，可增加外部高速 DAC 模块，将数字信号转换为真实的高频模拟信号输出，或利用千兆以太网（RMII/GMII）替代 UART，实现更高带宽的数据传输；二是存储与运算扩展，可增加双口 RAM 深度以支持超长任意波形，并引入 FFT 模块实现频谱分析；三是上位机功能深化，开发更复杂的通信协议解调功能与自动化测试脚本，使其更贴近工业级仪器标准。

4.2 心得体会

在本次项目的设计与研发过程中，我们团队经历了从基础模块设计、通信链路打通，到多模式扩展与系统时序收敛的完整开发周期。我们不仅完成了预期的功能，也通过解决一系列实际的工程问题，积累了宝贵的 FPGA 开发经验。

第一阶段：基础波形生成与仿真验证。在项目初期，我们基于 32 位相位累

加器与 1024 点 ROM 搭建了 DDS 基础底座。在这一阶段，我们遇到了两个典型问题。首先是波形显示异常，起初在仿真时方波会显示为三角波，经过排查确认这并非逻辑错误，而是 Vivado 仿真波形默认采用了线性插值（Linear Interpolation），将离散的跳变点连接成了斜线；通过在波形属性中将插值模式修改为“Hold”后，方波显示恢复正常。其次，我们遇到了 IP 缓存不更新的问题，修改 COE 文件后仿真结果不变，最终通过彻底删除 .sim 仿真缓存文件夹并重新编译解决了该问题。

第二阶段：上位机开发与通信链路联调。在将 FPGA 与 PC 端联合调试时，通信带宽成为了瓶颈。最初采用 115200 的波特率，导致回传采样率过低，上位机波形严重失真，输出波形可调频率范围过小。我们将波特率提升至 921600 以满足 20kHz 的采样需求，但随后引发了 Python 软件界面卡顿的问题。通过分析，我们采用了多线程架构，将串口读取与 UI 渲染解耦，并限制了单次读取的字节数，这样才保障了界面的流畅运行。

第三阶段：多模式扩展与时序收敛。在后期加入 LFSR 跳频、AM/FM 调制以及手绘任意波形（AWG）时，系统的复杂度和资源占用显著增加。在频率控制字的更新逻辑中，为了保证切换频率时相位不跳变且避免毛刺，我们在代码中加入了同步使能信号 ftw_valid，严格使用非阻塞赋值进行锁存。在综合与实现阶段，工程一度出现了时序违例，我们分析 Timing Summary 后发现是用于调试的 ILA 探针引入了过长的布线延迟，于是在 XDC 约束中添加了专门的忽略路径，最终让系统在 100MHz 时钟下实现了时序收敛。

第四阶段：工程可复现性与 AI 协作总结。为了满足比赛的可复现要求，我们编写了 Tcl 自动重建脚本。在此过程中，我们解决了相对路径解析错误、IP 参数版本兼容性等问题，最终实现了一键部署。

此外，在项目开发中，我们使用大模型辅助生成了部分基础代码框架与界面模板。但在底层硬件逻辑的调试中，我们认识到 AI 缺乏对物理延迟和时序约束的直觉，在实际工程中遇到复杂的时序逻辑中，AI 生成的组合逻辑代码容易产生毛刺，最终仍需工程师结合仿真波形与时序报告进行手动修正。

总体而言，本次比赛锻炼了我们在数字逻辑设计、软硬件协同调试以及问题排查等多方面的综合能力，是一次从理论仿真走向实际工程的全面实践。

第五部分 参考文献

[1] 夏宇闻. Verilog 数字系统设计教程[M]. 北京航空航天大学出版社, 2017.

[2] 正点原子. ATK-达芬奇 FPGA 开发指南[M]. 2023.

[3] 赛灵思 (Xilinx). Vivado Design Suite User Guide: Synthesis (UG901)[Z]. 2018.

[4] 赛灵思 (Xilinx). 7 Series FPGAs Memory Resources User Guide (UG473)[Z]. 2018.

[5] 加德纳 (Gardner, F. M.). 锁相环技术[M]. 北京: 人民邮电出版社, 2007.